

API Testing & Quality Assurance

UniTreeGroup Backend Documentation

Overview

The UniTreeGroup backend implements a multi-layered testing strategy designed to ensure financial accuracy, data integrity, and system resilience. Given the sensitive nature of financial stokvel operations, the testing suite focuses on atomicity, double-entry consistency, and concurrency protection.

Testing Stack

- **Test Framework:** xUnit
- **Mocking/Database:** Microsoft.Data.Sqlite (In-Memory mode for isolated relational testing)
- **Assertion Library:** xUnit standard assertions
- **Dependency Injection:** Microsoft.Extensions.DependencyInjection (to simulate real service resolution)

Core Test Suites

1. Financial Integrity Tests

File: Tests/FinancialFlowTests.cs

These tests ensure that the "Accounting Engine" follows the rules of double-entry bookkeeping:

- **Balanced Ledger Check:** Every transaction must create exactly two ledger entries (one Debit, one Credit) that sum to zero.
- **Balance Propagation:** Ensures that a Contribution correctly decrements the User's Wallet and increments the Group's Pool Balance.

- **Database Persistence:** Validates that data is correctly serialized/deserialized from the database using a fresh SQLite instance for every test.

2. Concurrency & Race Condition Tests

File: `Tests/FinancialFlowTests.cs`

Validates the system's ability to handle high-frequency, simultaneous requests:

- **Optimistic Concurrency (RowVersion):** Tests simulate two identical contribution requests hitting the service at the same time.
- **Expectation:** The system must throw an exception (e.g., `DbUpdateConcurrencyException`) for the second request, preventing "Double Spending" or data corruption.

3. Service-Level Validation

Validates business rules before any database interaction:

- **Membership Guards:** Ensuring users cannot join a group if they are already a member of another.
- **Insufficient Funds:** Verifying that a withdrawal or contribution fails if the wallet balance is too low.

How to Run the Tests

Command Line

Run all tests from the project root:

```
dotnet test
```

Filtering Tests

To run only the financial integrity tests:

```
dotnet test --filter FullyQualifiedName~FinancialFlowTests
```

API Endpoint Testing (Manual)

Swagger UI

The interactive documentation is available at `https://localhost:{port}/swagger` .

1. **Authorize:** Obtain a JWT from `/api/Auth/login` .
2. **Inject:** Click the "Authorize" button and enter `Bearer {your_token}` .
3. **Execute:** Test endpoints like `/api/Transactions/contribute` directly from the browser.

Postman / Insomnia

For complex testing scenarios, use the following collection structure:

- **Base URL:** `https://localhost:{port}/api`
- **Headers:** `Authorization: Bearer {{token}}`
- **Payloads:** Use the DTO structures defined in the `Dtos/` folder.

Financially accurate, resilient, and fully auditable.